Timed Technical Lab Writing

A workflow for writing technical documentation quickly and authentically under lab time constraints.

The core principle: **Completeness beats perfection under time pressure.** A finished, readable draft written in authentic student voice is more valuable than a polished paragraph that took twice as long.

Quick Overview

You will write three different types of technical documents in a single session, each with different requirements and time budgets:

Exercise	Format	Time	Audience	Output
User Manual	Step-by-step procedure	20 min	First-semester student (no experience with tool)	9-10 numbered steps + prereqs + screenshot description + troubleshooting note
API Reference	Technical reference entry	25 min	Developer (needs to call an API)	Endpoint spec + parameters + headers + response codes + JSON examples
Technical Report	Analysis/evaluation section	~30 min	Academic/professional audience	2-3 page section analyzing a system, tool, or concept

After writing, you'll participate in **peer critique**, where feedback is as important as the document itself.

Exercise 1: User Manual Procedure (20 minutes)

What You're Writing

A step-by-step guide for someone with basic computer skills but **no experience with your chosen tool**. They should be able to follow it without getting stuck.

Structure (in order)

1. **Title** — Clear, specific (e.g., "Setting Up a GitHub Repository and Making Your First Commit")
2. **Prerequisites** — Bulleted list of what they need before starting
3. **Steps** — 7-10 numbered steps, each with:
    - **One action per step** (not "download Git and open terminal" — that's two steps)
    - **Expected result** after each step (italicized) — "What you'll see" is clearer than "Expected Result"
4. **Screenshot description** — Describe what one screenshot would show at a key moment (you don't insert the actual image, just describe it)
5. **Troubleshooting note** — Address the single most common beginner error

Voice

- **Direct and conversational.** "Open your terminal" not "The user should then proceed to open a terminal application."
- **Vary sentence length.** Mix short punchy sentences with longer explanations.
- **Be specific about outcomes.** Don't say "the process completes." Say "Git prints 'Initialized empty Git repository...'"
- **Use "you" naturally.** "You'll see" is fine. "One may observe" is not.

Time-Saving Tips

- Don't explain *why* each step matters — just explain *how* to do it
- Use present tense throughout for consistency
- Copy-paste command examples exactly (e.g., `git init`, `cd Desktop/my-project`)
- Prerequisites go fast — just list 5-7 items with 1-2 word descriptions
- Troubleshooting: pick ONE error, not five edge cases

Common Trap

Rewriting steps multiple times to be "perfect." Write it once, move on. Your peer reviewers will give feedback.

Exercise 2: API Reference Entry (25 minutes)

What You're Writing

Documentation for a single API endpoint. A developer should be able to understand what it does, how to call it, and what might go wrong.

Structure (in order)

1. **Title/Endpoint** — "POST /api/v1/projects/{projectId}/tasks — Create Task"
2. **What it does** — 2-3 sentences in plain language (not corporate-speak)
3. **Request parameters** — Organized by:
    - Path parameters (in the URL)
    - Query parameters (if any)
    - Body parameters (in the JSON)
    - For each: name, data type, required/optional, description, example
4. **Required headers** — Authorization, Content-Type, etc.
5. **Response codes** — HTTP status codes (201, 400, 401, etc.) with plain-English explanations
6. **Example request** — Valid JSON showing what to send
7. **Example response** — Valid JSON showing what comes back on success

Voice

- **Avoid buzzwords.** Not "This endpoint allows authenticated users to leverage task management capabilities." Say "This endpoint lets you create a new task."
- **Be precise about data types.** "string, required" is clearer than "text field, must be filled"
- **Keep response code explanations short.** 1-2 sentences each.
- **Use realistic examples.** Not "user_1" — use "user_67890xyz" so it looks like real IDs.

Time-Saving Tips

- Don't overthink the API design — you're making it up, so be internally consistent and move on
- Response codes: 201 (success), 400 (bad data), 401 (auth), 403 (permission), 404 (not found), 500 (server error) covers 90% of cases
- JSON examples: keep them short but realistic (3-5 fields in request, 7-10 in response)
- Copy-paste JSON structure from your examples to reduce typing

Common Trap

Getting bogged down in "is this the right API design?" There's no single right answer — just be consistent.

Exercise 3: Technical Report Section (~30 minutes)

What You're Writing

An analytical section (2-3 pages) that evaluates, describes, or analyzes a technical system, tool, or concept. Your instructor will specify the topic.

Structure (common pattern)

1. **Introduction** — What is this thing? Why does it matter?
2. **How it works / Main features** — Describe the key components or functions
3. **Strengths / Use cases** — What's good about it? When would you use it?
4. **Limitations / Challenges** — What's not great? What are the trade-offs?
5. **Conclusion** — Summary or implications

Voice

- **Write like you're explaining to a peer.** Not like a Wikipedia article.
- **Include specific examples.** Reference real products, real data, real scenarios.
- **Acknowledge uncertainty.** "It's unclear whether..." is honest. Hedging everything ("arguably," "it could be said") reads as uncertain.
- **Show your thinking.** "This is interesting because..." or "The trade-off here is..." signals analysis, not just description.

Structure Tips

- Open with a concrete example or question (not a definition)
- Use headings and white space — it's easier to follow and takes less time to write
- Include at least one specific statistic, quote, or reference (even if paraphrased from memory)
- End with a clear takeaway, not vague hand-waving

Time-Saving Tips

- Spend 5 minutes outlining (intro, 2-3 main points, conclusion)
- Write the body sections first (you know them better) — intro and conclusion come faster at the end
- Don't fact-check everything — if you're uncertain about a detail, note it ("I think X, but I'm not certain")
- Use shorter paragraphs (3-4 sentences) so pages feel less dense

Common Trap

Trying to sound formal/academic. You're a Grade 11 student analyzing something — write like that. Specific and thoughtful beats flowery.

Authentic Voice (Humanizer Principles)

Your writing should sound like you, a Grade 11 student, explaining something clearly — not like ChatGPT.

What to Avoid

- Opening with "In conclusion" or "To summarize" at the end (just summarize)
- Using three synonyms in a row ("vital, crucial, essential")
- Phrases like "It is important to note that..." or "In order to..."
- Vague attributions ("Studies show..." without naming a study)
- Em dashes everywhere (—)
- Corporate tone ("leveraging," "facilitate," "optimize")

What to Do

- **Be specific.** Not "the tool is helpful" but "it saved me 10 minutes on this task"
- **Vary sentence structure.** Short. Then longer ones that take a breath. Mix it up.
- **Use "you" and "I" naturally.** Not "One might observe" — "You can see" or "I noticed"
- **Acknowledge reality.** "This didn't work as expected" is more honest than "There were unforeseen challenges"
- **Show your work.** "This confused me until I realized..." is better than vague praise

Test It

Read your writing aloud. If it sounds stiff or formal, rewrite it simpler. If you sound rushed or incomplete, that's OK — done is better than perfect under time pressure.

The Peer Critique (After Writing)

Critique is half the value of this lab. Here's how to make it count:

When Giving Feedback

- **Be specific.** Not "this step is confusing" but "I got stuck at Step 4 because I didn't know whether to use HTTPS or SSH"
- **Separate clarity from preference.** "This could be clearer" vs. "I would have written it differently" are different things
- **Suggest one concrete fix if you can.** "The screenshot description could mention what the error message says"

When Receiving Feedback

- **Don't defend your choices.** If someone was confused, they were confused — that's data

- **Ask for clarification.** "Can you tell me which part wasn't clear?" helps more than "I thought that was clear"
- **Look for patterns.** If two people got stuck at the same place, that's a signal

Pacing Checklist

User Manual (20 min)

- ☐ 0-2 min: Choose your tool, outline prerequisites
- ☐ 2-10 min: Write steps 1-5 (first half, usually fastest)
- ☐ 10-16 min: Write steps 6-9, screenshot description
- ☐ 16-19 min: Write troubleshooting note
- ☐ 19-20 min: Read once, catch obvious typos

API Reference (25 min)

- ☐ 0-2 min: Outline endpoint (method, path, what it does)
- ☐ 2-8 min: Write request parameters and headers
- ☐ 8-15 min: Write response codes (do these fast — one sentence each)
- ☐ 15-20 min: Write JSON examples (copy your parameter descriptions as a base)
- ☐ 20-24 min: Read once, verify JSON is valid syntax
- ☐ 24-25 min: Polish descriptions if time allows

Technical Report (~30 min)

- ☐ 0-3 min: Outline (intro, 2-3 main points, conclusion)
- ☐ 3-20 min: Write main sections (don't overthink phrasing)
- ☐ 20-27 min: Write intro and conclusion (fast because you know the content now)
- ☐ 27-30 min: Read once, fix flow issues and obvious errors

Quick Reference: Structure Templates

User Manual Step Template:

Step N: [One action]

[Description of what to do in 2-3 sentences. Include specific commands/names.]

What you'll see: [Describe the expected output or result.]

API Parameter Template:

fieldName (data_type, required/optional) — [1 sentence explaining what it is]. Example: "value_here"

API Response Code Template:

XXX Code Name — [1-2 sentences explaining when/why this happens.]

Key Principles

1. **Completeness > Perfection.** A finished, readable draft beats a polished first paragraph.
2. **Specific > Vague.** Name the tool, show the command, describe what appears on screen.
3. **Honest voice > Corporate voice.** Write like a student, not a press release.
4. **Done is better > Perfect is never.** Under time pressure, "done" wins.
5. **Feedback is fuel.** Peer critique teaches you more than finishing alone.

Common Questions

Q: Can I use bullet points in the user manual?
A: Yes, for prerequisites and lists. But steps should be numbered sentences — each step is one action with an outcome.

Q: Should I test the procedure / API I'm documenting?
A: If you can (for your chosen tool), yes. But the assignment is about writing documentation, not running the full process. A well-written procedure based on your knowledge is enough.

Q: How detailed should the API examples be?
A: Realistic but concise. Show all required fields and 1-2 optional ones. Real-world examples are better than placeholders.

Q: What if I don't finish?
A: That's OK. Write as much as you can in the time given. The critique process often reveals what needs revision anyway. Completing 80% of a clear draft is better than 40% of a polished one.

Q: Is my report supposed to be opinionated?
A: Yes and no. Be analytical (take a position: "X is better than Y because...") but support it with evidence or clear reasoning, not just opinion.